ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO THỰC TẬP

NGÀNH: CÔNG NGHỆ THÔNG TIN

ĐỀ TÀI: PHÁT TRIỂN VÀ TỐI ƯU TÍNH NĂNG CHO HỆ THỐNG QUẢN LÝ KHÁCH SẠN KIOTVIET HOTEL

Cán bộ hướng dẫn: Nguyễn Mạnh Cường

Giảng viên đánh giá: ThS. Trần Mạnh Cường

Sinh viên: Trần Quang Thành

Mã sinh viên: 23020155

Lớp: K68I-IT2

Hà Nội, tháng 9 năm 2026

MỤC LỤC

I. Giới thiệu chung 1
- a. Giới thiệu công ty 1
- b. Giới thiệu công việc 1
- c. Giới thiệu qua bài toán 1

II. Yêu cầu bài toán 2
- a. Miêu tả chi tiết bài toán 2
- b. Phần bài toán em trực tiếp giải quyết 3
- c. Công việc của nhóm và công việc của em 3

III. Tóm tắt lý thuyết, giải pháp, thuật toán 4
- a. Các lý thuyết và giải pháp liên quan 4
- b. Cách giải quyết của em 6
- c. Liên hệ và so sánh với các cách đã có 10

IV. Mô tả phần mềm cài đặt 11
- a. Phạm vi trình bày 11
- b. Môi trường và công cụ cần có 11
- c. Cách tổ chức mã nguồn và cách dựng phần mềm 11
- d. Quản lý thay đổi cơ sở dữ liệu 12
- e. Các môi trường và quy trình phát hành 12
- f. Kiểm thử tự động trong quy trình 13

V. Kết quả đạt được, hướng phát triển 13
- a. Kỹ năng và kiến thức thu thập được 13
- b. Hướng phát triển tiếp theo 16

Tài liệu tham khảo 17

LỜI CẢM ƠN

Thời gian làm việc tại KiotViet là quãng thời gian quý báu giúp em tích lũy được nhiều kinh nghiệm thực tế trước khi hoàn thành chương trình đại học. Em xin cảm ơn công ty KiotViet đã tạo điều kiện cho em được tiếp xúc, học hỏi và làm việc trong một môi trường chuyên nghiệp, năng động. Em xin gửi lời cảm ơn chân thành tới anh Nguyễn Mạnh Cường, người trực tiếp hướng dẫn em tại công ty. Anh luôn gợi mở và hướng dẫn em cách tự tìm hiểu bản chất của vấn đề thay vì đưa ra lời giải ngay. Những lần anh yêu cầu em tự giải thích và kiểm tra lại thay đổi trước khi đưa lên hệ thống đã giúp em hình thành thói quen cẩn trọng và có trách nhiệm hơn với công việc của mình.

Em cũng xin cảm ơn các anh chị trong nhóm phát triển sản phẩm khách sạn. Cảm ơn các anh chị đã luôn nhiệt tình hỗ trợ một người mới, kiên nhẫn chỉ ra những điểm chưa tối ưu trong mã nguồn và sẵn sàng chia sẻ kinh nghiệm thực tế giúp em tiến bộ nhanh hơn trong công việc.

Em xin cảm ơn các thầy cô Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội. Những kiến thức nền tảng về cơ sở dữ liệu, cấu trúc dữ liệu và giải thuật cùng công nghệ phần mềm được học tại trường đã giúp em tiếp cận công việc thực tế một cách thuận lợi và vững vàng hơn.

Cuối cùng, em xin gửi lời cảm ơn sâu sắc tới thầy Trần Mạnh Cường vì đã dành thời gian định hướng và hướng dẫn cho em hoàn thành học phần thực tập doanh nghiệp này. Báo cáo khó tránh khỏi những thiếu sót, em rất mong nhận được những ý kiến đóng góp của thầy để báo cáo được hoàn thiện hơn.

Em xin chân thành cảm ơn!

Hà Nội, tháng 9 năm 2026

Sinh viên

Trần Quang Thành

I. Giới thiệu chung

a. Giới thiệu công ty

KiotViet là giải pháp phần mềm quản lý bán hàng và vận hành doanh nghiệp phổ biến tại Việt Nam, do Công ty Cổ phần Công nghệ Citigo nghiên cứu và phát triển. Xuất phát từ định hướng hỗ trợ các hộ kinh doanh và doanh nghiệp vừa và nhỏ chuyển đổi số, thay thế sổ tay ghi chép bằng phần mềm quản trị, KiotViet đã mở rộng từ bán lẻ sang nhà hàng, cà phê, làm đẹp và lưu trú khách sạn. Mỗi ngành có đặc thù riêng nên được tổ chức thành các dòng sản phẩm chuyên sâu dùng chung hạ tầng dịch vụ nền tảng.

Trong kỳ thực tập, em tham gia nhóm phát triển KiotViet Hotel, giải pháp chuyên biệt cho cơ sở lưu trú từ homestay, nhà nghỉ đến khách sạn và resort. Người dùng trực tiếp gồm lễ tân, thu ngân, buồng phòng, kế toán và chủ cơ sở. Bên cạnh thị trường trong nước, sản phẩm còn phục vụ khách hàng tại một số quốc gia khác trong khu vực.

b. Giới thiệu công việc

Em tham gia nhóm dự án với vai trò thực tập sinh phát triển phần mềm, trực tiếp tham gia lập trình cả tầng giao diện (Frontend) lẫn phía máy chủ (Backend).

Thời gian đầu, em không khỏi bỡ ngỡ trước quy mô của hệ thống. Khác với các bài tập lớn ở trường, sản phẩm được xây dựng từ nhiều thành phần khác nhau, từ màn hình lễ tân, cổng quản trị đến các dịch vụ và thư viện dùng chung. Với hàng trăm nghìn dòng mã sẵn có, việc đầu tiên của em là đọc hiểu luồng dữ liệu: khi lễ tân bấm một nút trên màn hình thì yêu cầu chạy qua những đâu.

Công việc của em tập trung vào bốn mảng chính: sửa lỗi từ vận hành thực tế, phát triển tính năng mới, tối ưu hiệu năng tại các điểm nghẽn và tham gia xây dựng phiên bản quản lý mới.

c. Giới thiệu qua bài toán

Phần mềm quản lý khách sạn thoạt nhìn có vẻ đơn giản: khách đến nhận phòng, lưu trú và thanh toán khi trả phòng. Nhưng khi bắt tay vào làm, em nhận thấy độ khó nằm ở việc duy trì tính nhất quán khi cùng một dữ liệu phải trả lời cho nhiều câu hỏi nghiệp vụ khác nhau, tại nhiều thời điểm khác nhau.

Một ví dụ thực tế em từng gặp là tính năng khóa phòng để sửa chữa. Nhìn từ giao diện, đây chỉ là thao tác chọn ngày, nhưng phía sau lại liên quan đến nhiều phần của hệ thống. Màn hình lễ tân phải lập tức ngăn việc đặt phòng, sơ đồ phòng phải hiển thị khoảng thời gian bị khóa, báo cáo công suất cũng phải phản ánh đúng trạng thái của phòng. Đồng thời, khi xem lại dữ liệu của tháng trước, phòng vẫn được tính bình thường

vì tại thời điểm đó chưa bị khóa. Qua đó, em nhận ra rằng một thay đổi tưởng như rất nhỏ trên giao diện cũng có thể ảnh hưởng đến nhiều phần khác nhau của hệ thống.

Bài toán còn phức tạp bởi ba ràng buộc: hệ thống vận hành 24/7 nên mọi sửa đổi phải chạy an toàn trên môi trường thật; khách hàng có nhiều chi nhánh ở các múi giờ và đồng tiền khác nhau; và giao diện hỗ trợ nhiều ngôn ngữ đòi hỏi xử lý chuẩn xác từ chuỗi chữ đến định dạng ngày và số. Trong bối cảnh đó, phần việc của em là tham gia bổ sung tính năng và hỗ trợ nâng cao độ tin cậy của hệ thống trong quá trình vận hành.

II. Yêu cầu bài toán

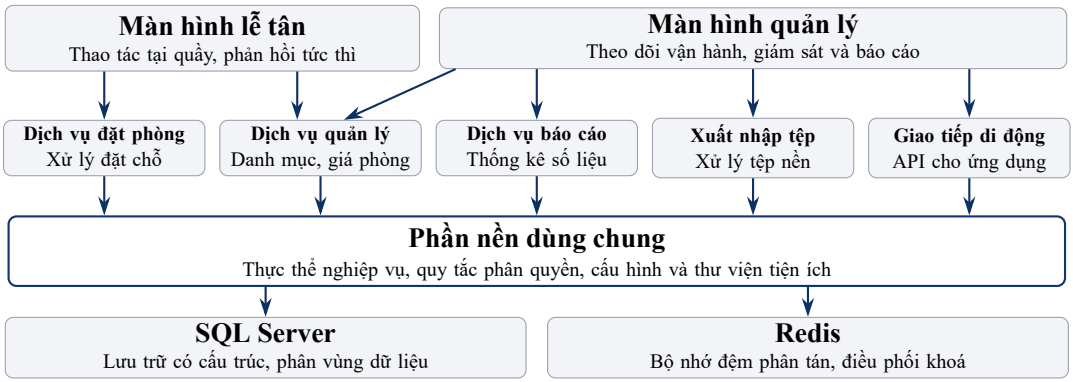
a. Miêu tả chi tiết bài toán

Hệ thống phục vụ ba nhóm người dùng có nhu cầu rất khác nhau, và chính sự khác biệt đó tạo nên phần lớn độ phức tạp trong thiết kế và tối ưu.

Lễ tân và thu ngân làm việc trực tiếp tại quầy vào các khung giờ cao điểm nhận và trả phòng, đòi hỏi thao tác nhanh, quy trình tinh giản và phản hồi tức thì: màn hình phải trả lời ngay phòng nào trống, khách đã thanh toán chưa, số tiền còn thiếu là bao nhiêu. Độ trễ dù chỉ vài giây tại quầy cũng tác động trực tiếp đến sự chờ đợi của khách lưu trú.

Người quản lý và chủ cơ sở quan tâm đến các con số tổng hợp: doanh thu so với kỳ trước, công suất phòng và hiệu quả nguồn khách. Các truy vấn này quét trên tập dữ liệu lớn với yêu cầu khắt khe về tính nhất quán: số liệu giữa báo cáo phân tích và màn hình theo dõi thời gian thực cần đảm bảo tính đồng bộ, không để xảy ra sai lệch. Nhóm thứ ba gồm bộ phận kế toán và buồng phòng với các nghiệp vụ cụ thể như xuất hóa đơn, đối chiếu sổ quỹ, bàn giao ca và cập nhật tình trạng dọn phòng.

Qua quá trình làm việc và tiếp cận hệ thống, em nhận thấy sản phẩm cần đáp ứng một số yêu cầu chính như: số liệu cần đảm bảo tính nhất quán giữa các màn hình nghiệp vụ; quyền thao tác của người dùng phải được kiểm tra ở phía máy chủ, thay vì chỉ ẩn hoặc vô hiệu hóa nút trên giao diện; hệ thống cần hỗ trợ địa phương hóa tốt về ngôn ngữ, múi giờ và tiền tệ; đồng thời cần duy trì hiệu năng ổn định, tránh để các tác vụ xuất nhập dữ liệu lớn gây ảnh hưởng đến thao tác thường nhật của nhân viên tại quầy.



Hình 1. Các thành phần chính của hệ thống và quan hệ giữa chúng

Hình 1 mô tả kiến trúc phân tầng và sự tương tác giữa các thành phần chính trong hệ thống. Trong quá trình thực tập, em nhận thấy các dịch vụ nghiệp vụ có mối liên kết chặt chẽ với nhau: một thao tác thực tế của người dùng thường đi qua nhiều tầng chức năng, do đó việc đọc hiểu mã nguồn hiện hữu là bước rất quan trọng trước khi bắt tay vào xây dựng tính năng mới.

b. Phần bài toán em trực tiếp giải quyết

Hệ thống sản phẩm là kết quả xây dựng của cả tập thể qua nhiều giai đoạn. Trong phạm vi thực tập, em được giao đảm nhiệm một số bài toán cụ thể với đầu vào và đầu ra rõ ràng, tập trung vào sáu hướng chính:

1. Tham gia xây dựng tính năng khóa phòng bảo trì: từ mô hình dữ liệu, quy tắc nghiệp vụ, API phía máy chủ đến giao diện người dùng, bản dịch đa ngôn ngữ và kiểm thử tự động.
2. Bổ sung hai chỉ tiêu thống kê về lượt khách nghỉ qua đêm và tổng số đêm nghỉ-bài toán xử lý khoảng thời gian và băm chuỗi định danh.
3. Tái cấu trúc phân quyền: tách quyền hủy đơn thành hai quyền độc lập, kèm kịch bản chuyển đổi dữ liệu phân quyền hai chiều cho các cơ sở đang hoạt động.
4. Tối ưu hiệu năng: cải tiến các câu truy vấn cơ sở dữ liệu, giải phóng khóa phân tán kịp thời và loại bỏ các phép tính toán lặp dư thừa trên giao diện.
5. Xây dựng các phân hệ cho bản quản lý mới: trang tổng quan (dashboard) với các biểu đồ doanh thu, danh mục phòng và tài liệu hướng dẫn tích hợp kênh bán phòng trực tuyến.
6. Xử lý đa ngôn ngữ và địa phương hóa: bổ sung tiếng Hàn, tiếng Trung, tiếng Lào cho các màn hình, đồng thời chuẩn hóa định dạng ngày tháng, tiền tệ và múi giờ.

c. Công việc của nhóm và công việc của em

Nhóm phát triển vận hành theo mô hình đa chức năng: chuyên viên phân tích nghiệp vụ đặc tả yêu cầu; thiết kế trải nghiệm dựng giao diện; lập trình viên hiện thực logic; chuyên viên kiểm thử (QA) kiểm tra chất lượng trước phát hành; hỗ trợ kỹ thuật tiếp nhận phản hồi từ khách hàng; và trưởng nhóm điều phối, rà soát mã nguồn cùng tiến độ.

Trong cấu trúc đó, em tham gia với vai trò thực tập sinh phát triển phần mềm. Quy trình xử lý một phiếu công việc bắt đầu từ việc tiếp nhận yêu cầu, trao đổi làm rõ trường hợp biên, đọc mã nguồn hiện hữu để xác định phạm vi ảnh hưởng, viết mã, tự kiểm thử,

bổ sung kiểm thử tự động cho logic quan trọng, gửi rà soát mã nguồn (code review) và phối hợp với bộ phận kiểm thử đến khi nghiệm thu hoàn tất.

Một bài học làm thay đổi nhận thức nghề nghiệp của em: nhiệm vụ thực sự của người phát triển không chỉ là làm cho tính năng chạy được, mà phải đảm bảo nó hoạt động chuẩn xác mà không gây ảnh hưởng tiêu cực đến bất kỳ hành vi nghiệp vụ nào đang vận hành ổn định trên hệ thống.

III. Tóm tắt lý thuyết, giải pháp, thuật toán

a. Các lý thuyết và giải pháp liên quan

Trong quá trình thực tập, em đã trực tiếp vận dụng các nền tảng của khoa học máy tính và nguyên lý thiết kế phần mềm vào quá trình phân tích, xây dựng và cải tiến hệ thống. Những kiến thức này được gắn với các bài toán thực tế và các giải pháp kỹ thuật được trình bày trong những phần tiếp theo.

- **Kiến trúc phân tầng và nguyên tắc đảo ngược phụ thuộc:** Hệ thống được tổ chức thành các tầng với trách nhiệm rõ ràng theo nguyên lý kiến trúc sạch (Clean Architecture) [1]: tầng giao diện xử lý tương tác với người dùng, tầng ứng dụng và nghiệp vụ xử lý các quy tắc của hệ thống, còn tầng hạ tầng đảm nhiệm việc làm việc với cơ sở dữ liệu và các thành phần bên ngoài. Các tầng nghiệp vụ cốt lõi được thiết kế để không phụ thuộc trực tiếp vào giao diện hay công nghệ lưu trữ cụ thể. Nhờ đó, các quy tắc nghiệp vụ quan trọng được tập trung và quản lý thống nhất, đồng thời có thể được sử dụng lại ở nhiều chức năng khác nhau mà không phải sao chép hoặc viết lại cùng một logic.

- **Phân tách lệnh và truy vấn (CQRS):** Hệ thống áp dụng mô hình phân tách lệnh và truy vấn (CQRS) [4] để tách riêng các thao tác làm thay đổi dữ liệu (Command) với các thao tác chỉ đọc và truy xuất dữ liệu (Query). Với các nghiệp vụ như đặt phòng, thay vì thay đổi trực tiếp trạng thái của bản ghi, hệ thống xử lý thông qua các lệnh thể hiện rõ mục đích của nghiệp vụ. Cách tiếp cận này giúp dễ dàng theo dõi và lưu lại lịch sử thay đổi, đồng thời cho phép phần xử lý dữ liệu đọc được tối ưu riêng cho các chức năng như báo cáo và thống kê mà không ảnh hưởng đến quá trình ghi dữ liệu.

- **Chi phí của tầng trừu tượng khi truy cập dữ liệu (ORM Overhead):** Các công cụ ánh xạ đối tượng-quan hệ (ORM) như Entity Framework [6] giúp lập trình viên thao tác với cơ sở dữ liệu thông qua ngôn ngữ bậc cao. Tuy nhiên, nếu sử dụng không phù hợp, tầng trừu tượng này có thể sinh ra các câu truy vấn phức tạp, nạp thừa dữ liệu qua mạng (over-fetching) hoặc dẫn đến hiện tượng bùng nổ tích

Descartes khi nối nhiều bảng quan hệ. Tại các tác vụ xử lý lượng bản ghi lớn, việc xem xét kế hoạch thực thi (Execution Plan) và cơ chế khóa [7] kết hợp sử dụng các câu truy vấn định hướng (Projection Queries) là yếu tố quan trọng để đảm bảo hiệu năng.

- **Khóa phân tán và quản lý vòng đời khóa (Distributed Lock with TTL):** Khi nhiều tiến trình cùng truy cập và cập nhật tài nguyên dùng chung, khóa phân tán trên Redis hỗ trợ xử lý tuần tự và hạn chế xung đột tài nguyên [8]. Việc thiết lập thời gian sống (TTL) cho khóa là điều cần thiết để giảm nguy cơ khóa bị giữ vô thời hạn (stale lock) trong trường hợp một tiến trình gặp sự cố bất thường trước khi kịp giải phóng khóa.
- **Cơ chế phát hiện thay đổi trên giao diện (Change Detection):** Trong Angular, quá trình phát hiện thay đổi có thể khiến các biểu thức trong template được đánh giá lại qua nhiều chu kỳ cập nhật giao diện [5]. Do đó, việc gọi trực tiếp các hàm tính toán phức tạp hoặc quét mảng trong template có thể bị kích hoạt lặp đi lặp lại, gây sụt giảm khung hình và làm giao diện phản hồi chậm chạp.
- **Thuật toán gộp khoảng và tính giao thời gian (Interval Merging):** Cho một tập các khoảng thời gian có khả năng chồng lấn $I = \{[s_i, e_i]\}$ [3], thuật toán tiến hành sắp xếp theo mốc bắt đầu với độ phức tạp $O(k \log k)$, sau đó duyệt tuần tự trong $O(k)$ để hợp nhất các khoảng có phần giao nhau thành các khoảng liên tục không chồng lấn. Từ các khoảng đã gộp, hệ thống tính toán phần giao thực tế với một khung thời gian quan sát $[T_{\text{start}}, T_{\text{end}}]$ cho trước.
- **Hàm băm một chiều bảo vệ dữ liệu định danh (One-way Hashing):** Nhằm tuân thủ các quy chuẩn bảo mật và an toàn thông tin cá nhân (PII), dữ liệu định danh của khách lưu trú được chuyển thành giá trị băm có độ dài cố định bằng hàm băm một chiều. Kỹ thuật này cho phép hệ thống đối chiếu và gom nhóm khách hàng dựa trên giá trị định danh mà không cần sử dụng trực tiếp thông tin gốc trên phân hệ báo cáo.
- **Chuẩn hóa chuỗi văn bản tìm kiếm đa ngôn ngữ (Unicode Normalization):** Xử lý tìm kiếm trong môi trường đa ngôn ngữ đòi hỏi chuẩn hóa cả chuỗi người dùng nhập lẫn dữ liệu nguồn về dạng chuẩn (như NFC). Đồng thời, thuật toán sử dụng các thuộc tính ký tự Unicode (`\p{L}`, `\p{N}`) theo chuẩn Unicode [9] thay cho các biểu thức chính quy thuần ASCII, nhằm bảo toàn trọn vẹn dấu tiếng Việt và các hệ ký tự tượng hình như tiếng Hàn và tiếng Trung.

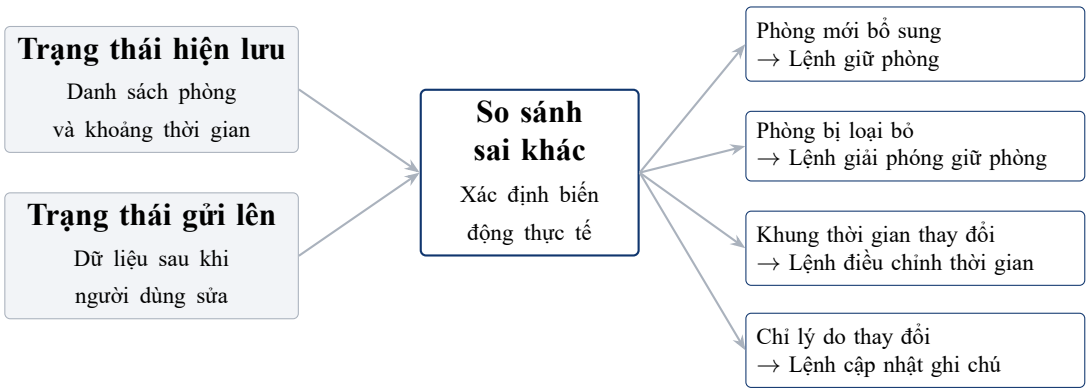
b. Cách giải quyết của em

Tính năng khóa phòng: xử lý thay đổi trạng thái

Tính năng khóa phòng phục vụ nhu cầu tạm ngừng kinh doanh một hoặc nhiều phòng trong một khoảng thời gian xác định kèm lý do bảo trì hoặc sửa chữa. Vấn đề phức tạp nhất xuất hiện ở thao tác chỉnh sửa: người dùng mở một lượt khóa hiện có, sau đó có thể đồng thời thêm phòng mới, loại bỏ phòng cũ, thay đổi khung thời gian bắt đầu/kết thúc và cập nhật lại nội dung lý do.

Phương án đơn giản nhất có thể nghĩ tới là xóa sạch toàn bộ lượt khóa cũ rồi tạo lại lượt mới từ đầu. Phương án này dễ lập trình nhưng bộc lộ hai nhược điểm: lịch sử biến động phòng bị xóa trắng khiến ban quản lý khó truy vết những thay đổi đã thực hiện; đồng thời, các phòng không hề bị thay đổi cũng phải trải qua chu kỳ hủy và tạo lại, gây ra các biến động trạng thái dư thừa và làm tăng nguy cơ xung đột khóa với các đơn đặt phòng khác đang diễn ra.

Để xử lý vấn đề này, em so sánh trạng thái phòng trước và sau khi chỉnh sửa để xác định chính xác những thay đổi thực tế cần xử lý. Cụ thể, gọi R_{old} là tập phòng đang khóa và R_{new} là tập phòng sau chỉnh sửa: các phòng thuộc $R_{\text{new}} \setminus R_{\text{old}}$ sinh lệnh giữ phòng mới; các phòng thuộc $R_{\text{old}} \setminus R_{\text{new}}$ sinh lệnh giải phóng; nếu khung thời gian thay đổi thì sinh lệnh điều chỉnh thời gian; và nếu chỉ sửa đổi lý do thì chỉ phát sinh lệnh cập nhật ghi chú. Thành phần xử lý sai khác này được thiết kế độc lập với cơ sở dữ liệu, giúp thuận tiện cho việc viết kiểm thử tự động với các kịch bản tổ hợp.



Hình 2. Chỉ sinh lệnh cho phần thực sự thay đổi thay vì huỷ và tạo lại toàn bộ

Hình 2 minh họa trực quan luồng phát sinh lệnh có chọn lọc. Toàn bộ các quy tắc kiểm tra tính hợp lệ của dữ liệu đầu vào cũng được tập trung tại một đầu mối duy nhất: loại khóa hợp lệ, giới hạn độ dài lý do, bắt buộc chọn ít nhất một phòng, không cho phép chọn trùng lặp và thời điểm kết thúc luôn phải sau thời điểm bắt đầu. Việc gom cụm quy tắc này bảo đảm thông báo lỗi đồng nhất trên toàn hệ thống.

Ở tầng giao diện, em xây dựng hộp thoại thao tác tích hợp vào sơ đồ lịch phòng và bổ sung cơ chế kiểm tra chặn khi đặt phòng. Đặc biệt, sau khi hoàn thành tính năng, em

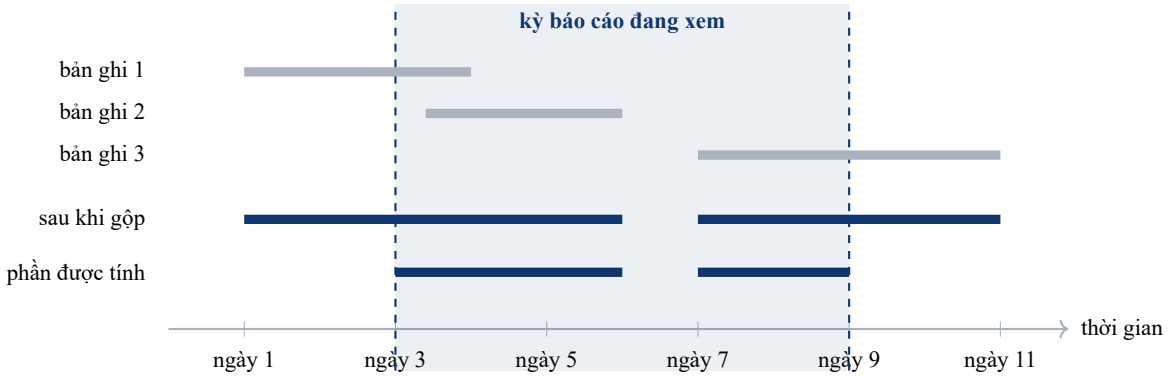
chủ động phân tích tác động dây chuyền và phát hiện các công thức tính tỷ lệ công suất phòng đang tính cả phòng bị khóa vào số phòng khả dụng để bán. Điều này khiến tỷ lệ lấp đầy hiển thị thấp hơn thực tế, làm sai lệch đánh giá hiệu quả kinh doanh của cơ sở. Em đã chủ động trao đổi và điều chỉnh lại công thức để loại trừ các phòng đang khóa bảo trì ra khỏi mẫu số tính công suất, giúp số liệu thống kê phản ánh chính xác thực tế vận hành.

Thống kê lượt khách nghỉ qua đêm và tổng số đêm nghỉ

Yêu cầu bài toán là bổ sung hai chỉ tiêu thống kê quan trọng vào báo cáo phòng trong một khoảng thời gian tùy chọn. Khi phân tích yêu cầu, em nhận thấy phần tính toán cần xử lý ba vấn đề chính:

1. Xử lý các khoảng lưu trú chồng lấn: Một khách hàng có thể có nhiều bản ghi lưu trú nối tiếp hoặc chồng lấn nhau trong kỳ do đổi phòng hoặc gia hạn phòng. Nếu cộng dồn số đêm của từng bản ghi, các ngày trùng lặp sẽ bị tính nhiều lần làm phồng số liệu. Em áp dụng thuật toán gộp khoảng [3]: sắp xếp toàn bộ các bản ghi của cùng một khách theo thời điểm nhận phòng, sau đó duyệt tuần tự để hợp nhất các khoảng có phần giao nhau thành các khoảng lưu trú liên tục duy nhất.

2. Cắt khoảng theo biên kỳ báo cáo: Một lượt lưu trú có thể bắt đầu trước hoặc kết thúc sau khoảng thời gian đang xét. Em xác định riêng quy tắc tính toán cho từng chỉ tiêu: số lượt khách chỉ được ghi nhận khi thời điểm nhận phòng nằm trong kỳ báo cáo (vì lượt là một sự kiện phát sinh); trong khi tổng số đêm nghỉ được tính bằng độ dài phần giao thực tế giữa các khoảng lưu trú đã gộp với khung thời gian báo cáo.



Hình 3. Gộp các khoảng lưu trú chồng lấn rồi cắt theo biên của kỳ báo cáo

Hình 3 mô tả hai bước xử lý: ba bản ghi ban đầu sau khi hợp nhất tạo thành hai khoảng rời rạc; khi đối chiếu theo khung báo cáo, các phần nằm ngoài khoảng thời gian báo cáo được loại bỏ. Bản ghi 1 bắt đầu trước kỳ nên không tính vào số lượt, nhưng các đêm nằm trong kỳ vẫn được tính đầy đủ vào tổng số đêm nghỉ.

3. Xử lý trường hợp định danh rỗng: Khách hàng được nhận diện thông qua giá trị băm của thông tin định danh cá nhân. Trong quá trình kiểm tra thực tế, em phát hiện

một vấn đề nghiệp vụ: đối với những khách vãng lai hoặc khách thuê theo giờ không khai báo giấy tờ tùy thân, chuỗi định danh ban đầu là chuỗi rỗng. Khi đi qua hàm băm, chuỗi rỗng luôn sinh ra cùng một giá trị băm cố định. Nếu gom nhóm đơn thuần theo giá trị băm này, toàn bộ khách không khai báo giấy tờ trong suốt kỳ báo cáo sẽ bị gộp thành một người duy nhất, dẫn đến số lượt khách hiển thị bị sai lệch nghiêm trọng. Để khắc phục, hệ thống phân biệt trường hợp không có định danh và quản lý các lượt lưu trú này dựa trên mã đặt phòng (booking code) kết hợp thời gian lưu trú cụ thể, nhờ đó mỗi lượt khách chưa khai báo giấy tờ vẫn được tính toán độc lập và chính xác theo đúng nghiệp vụ.

Toàn bộ logic tính toán được đóng gói thành một lớp thuần túy độc lập với cơ sở dữ liệu, cho phép thiết lập bộ kiểm thử bao phủ các tình huống biên phức tạp. Trong đó, em bổ sung trường hợp kiểm thử kiểm tra tính chất nhất quán của thuật toán: tổng kết quả tính toán của hai khoảng thời gian liên tiếp ghép lại phải bằng đúng kết quả khi tính một lần trên toàn bộ khoảng thời gian gộp.

Tối ưu quá trình xuất dữ liệu và xử lý các vấn đề về hiệu năng

Chức năng xuất tệp sổ quỹ gặp tình trạng chậm khi dữ liệu lớn. Phân tích mã nguồn cho thấy phần lấy tên phòng được xây dựng bằng nhiều biểu thức lồng nhau, thực hiện phép nối trên nhiều bảng và lọc trùng lặp qua nhiều bước. Tuy nhiên, điểm nghẽn lớn nhất nằm ở thuật toán xử lý dữ liệu sau khi nạp: mã nguồn cũ duyệt qua từng dòng của tệp và quét tuần tự toàn bộ danh sách phòng để tìm kiếm tên tương ứng. Với tệp n dòng và danh sách phòng m phần tử, số phép so sánh có độ phức tạp $O(n \times m)$, gây độ trễ đáng kể khi xuất hàng nghìn bản ghi.

Em tái cấu trúc thuật toán theo ba bước: kiểm tra sớm nếu không có dữ liệu cần xuất thì dừng ngay; chuyển đổi chuỗi biểu thức phức tạp thành một câu lệnh truy vấn chuẩn hóa duy nhất ở tầng cơ sở dữ liệu để lấy đủ thông tin cần thiết; và chuyển danh sách phòng thành một bảng tra cứu (Hash Map) theo khóa định danh. Thuật toán mới giảm độ phức tạp xuống $O(n + m)$, giảm số vòng lặp và làm luồng xử lý đơn giản hơn, giúp thời gian xử lý khi xuất dữ liệu lớn được cải thiện rõ rệt.

Một tình huống khác liên quan đến tranh chấp tài nguyên cơ sở dữ liệu khi người dùng xuất báo cáo hóa đơn chi tiết. Thay vì can thiệp vào cơ chế quản lý giao dịch chung vốn tiềm ẩn nhiều rủi ro, em phân tích câu truy vấn và nhận thấy hệ thống đang nạp kèm một số dữ liệu quan hệ dư thừa mà mẫu tệp không sử dụng. Việc nạp dữ liệu không cần thiết làm câu truy vấn phức tạp hơn, tăng lượng dữ liệu xử lý và kéo dài thời gian thực thi, từ đó làm tăng khả năng tranh chấp tài nguyên trên cơ sở dữ liệu [7]. Việc loại bỏ các quan hệ nạp thừa và tối ưu danh sách truy vấn đã giúp xử lý được hiện tượng nghẽn này mà chỉ cần điều chỉnh một phần nhỏ mã nguồn.

Ngoài ra, trong quá trình rà soát, em nhận thấy một số khóa phân tán trên Redis được tạo ra nhưng chưa thiết lập thời gian sống (TTL) [8]. Nếu tiến trình xử lý gặp sự cố bất thường trước khi kịp giải phóng, khóa có thể bị giữ lại và gây ảnh hưởng đến các tiến trình tiếp theo. Em đã bổ sung cơ chế TTL tự động giải phóng khóa, giúp giảm thiểu nguy cơ tài nguyên bị chiếm dụng vô thời hạn.

Tập trung quy tắc phân quyền vào một chỗ

Yêu cầu đặt ra là phân tách quyền hủy đơn phòng cũ thành hai quyền hạn độc lập dựa trên trạng thái lưu trú của khách. Bài toán có hai khía cạnh cần xử lý đồng bộ:

Về mặt dữ liệu, hàng nghìn khách hàng đang vận hành đã được gán quyền hạn theo cấu trúc cũ. Việc chuyển đổi sang hai quyền mới phải đảm bảo tính liên tục, không làm gián đoạn hay mất mát quyền của bất kỳ tài khoản nào. Em xây dựng kịch bản chuyển đổi dữ liệu tự động, đồng thời viết kèm kịch bản khôi phục (rollback) để đưa dữ liệu về trạng thái ban đầu nếu phát sinh sự cố. Kịch bản dự phòng là nguyên tắc bắt buộc khi can thiệp vào dữ liệu sản xuất.

Về mặt giao diện, logic kiểm tra quyền xuất hiện tại nhiều vị trí trên màn hình quầy. Thay vì rải các câu điều kiện kiểm tra ở từng nút bấm, em đóng gói toàn bộ quy tắc vào một dịch vụ quản lý quyền chuyên biệt. Các thành phần hiển thị chỉ cần gọi một phương thức duy nhất để xác định quyền thao tác của người dùng hiện tại. Đồng thời, quyền thao tác của người dùng phải được kiểm tra ở phía máy chủ, bởi việc ẩn hoặc vô hiệu hóa nút trên giao diện chỉ đóng vai trò hỗ trợ trải nghiệm chứ không thể thay thế cho việc kiểm tra quyền thực tế.

Giữ cho giao diện nhẹ và tách logic khỏi phần hiển thị

Quy ước của nhóm dự án là hạn chế tối đa việc gọi các hàm hoặc thuộc tính tính toán trực tiếp trong template của Angular [5]. Em áp dụng hai giải pháp: gán trước kết quả tính toán vào biến trạng thái khi dữ liệu thay đổi để giao diện chỉ việc đọc giá trị; và sử dụng các pipe thuần túy (pure pipes) có cơ chế lưu kết quả để tái sử dụng (memoization) [5] cho các bảng dữ liệu danh sách dài, giúp hệ thống chỉ tính toán lại khi tham số đầu vào thực sự biến động.

Tương tự, khi xây dựng ô tìm kiếm cho tài liệu hướng dẫn tích hợp kênh bán phòng trực tuyến, em tách riêng logic xử lý chuỗi ra một lớp tiện ích gồm ba bước: chuẩn hóa văn bản, tách mảng từ khóa và đối chiếu nội dung. Khi chuẩn hóa, em sử dụng bộ nhận diện ký tự Unicode thay vì lọc theo bảng chữ cái ASCII nhằm bảo toàn dấu tiếng Việt và chữ tượng hình [9]; khi đối chiếu, hệ thống yêu cầu tiêu đề chứa đầy đủ các từ khóa mà không bắt buộc theo đúng thứ tự gõ.

Dựng lại biểu đồ doanh thu và một lỗi lặp lại ba lần

Khi xây dựng lại màn hình quản lý tổng quan, em tiếp nhận thành phần biểu đồ doanh thu cũ vốn trộn lẫn logic nạp dữ liệu, tính toán thống kê và dựng đồ họa trong một tệp duy nhất. Em đã tái cấu trúc thành ba thành phần độc lập, chuyển phần giao tiếp dữ liệu về đúng tầng dịch vụ chuyên trách.

Trong quá trình hoàn thiện, ba phiếu lỗi riêng biệt xuất hiện trên cùng màn hình: bỏ chọn một chi nhánh nhưng tổng doanh thu không đổi, biểu đồ cơ cấu nguồn khách không vẽ lại, và khối tổng tiền phòng giữ nguyên giá trị cũ. Qua việc kiểm tra ba lỗi trên, em nhận thấy chúng đều bắt nguồn từ cùng một nguyên nhân: các thành phần con tự lưu trữ bản sao danh sách chi nhánh riêng biệt, dẫn đến sự lệch pha khi có sự kiện cập nhật. Em khắc phục bằng cách chuyển sang chia sẻ một nguồn dữ liệu trạng thái chi nhánh chung (Single Source of Truth). Sau khi thống nhất nguồn dữ liệu trạng thái ở tầng dịch vụ dùng chung, ba trường hợp lỗi trên không còn xuất hiện trong quá trình kiểm thử. Bài học rút ra là khi một hiện tượng xuất hiện ở nhiều nơi, việc tìm đúng nguyên nhân gốc trong thiết kế luồng dữ liệu sẽ giúp xử lý vấn đề hiệu quả hơn so với việc sửa riêng lẻ tại từng giao diện.

c. Liên hệ và so sánh với các cách đã có

Trong phạm vi công việc được giao, em chủ yếu so sánh giải pháp được lựa chọn với các phương án đã cân nhắc trong quá trình phát triển thực tế tại doanh nghiệp, thay vì so sánh với các sản phẩm bên ngoài.

Về giải pháp tính năng khóa phòng: Phương án ban đầu có thể nghĩ tới là xóa sạch lượt khóa cũ rồi tạo lại, ưu điểm là mã nguồn ngắn và triển khai nhanh. Tuy nhiên, giải pháp so sánh trạng thái trước và sau khi chỉnh sửa mà em áp dụng mang lại nhiều ưu thế hơn: lưu giữ được đầy đủ nhật ký thao tác cho người quản trị; giảm thiểu biến động trạng thái trên hệ thống; và hạn chế nguy cơ xung đột khóa dữ liệu với các đơn đặt phòng khác đang diễn ra.

Về vị trí đặt phần tính toán thống kê: Ban đầu, em đặt module tính toán tại dịch vụ quản lý do báo cáo được mở từ giao diện này. Sau khi trao đổi cùng cán bộ hướng dẫn, em nhận ra cách tiếp cận đó chưa hợp lý: dữ liệu lưu trú thuộc trách nhiệm quản lý của dịch vụ đặt phòng, việc để dịch vụ quản lý tự đọc và tính toán sẽ tạo ra hai nơi cùng định nghĩa một chỉ tiêu nghiệp vụ. Em đã chủ động hủy bỏ phần mã cũ để chuyển toàn bộ logic tính toán về dịch vụ đặt phòng. Nguyên tắc em rút ra là trong kiến trúc của hệ thống, nhóm ưu tiên đặt logic xử lý tại dịch vụ sở hữu dữ liệu nguồn.

Về phạm vi tác động của một thay đổi: Khi phát hiện lỗi định dạng ngày trên giao diện tiếng Hàn, em nhận thấy mã nguồn xử lý ngày tháng cũ bị phân tán và ban đầu đã viết lại hơn một trăm dòng mã trên nhiều tệp khác nhau. Tuy nhiên, nhận thấy việc

thay đổi trên diện rộng tiềm ẩn rủi ro khi chưa có đầy đủ kiểm thử tự động bao phủ toàn bộ các màn hình, em đã hủy bỏ thay đổi đó và chuyển sang giải pháp an toàn hơn: chỉ tối ưu hàm đọc ngày tháng tại đúng điểm phát sinh lỗi trong phạm vi khoảng hai mươi dòng mã. Bài học em rút ra là việc tái cấu trúc mã nguồn trên diện rộng cần được hoạch định độc lập với bộ kiểm thử bảo vệ [2], không nên tùy tiện gộp chung vào một phiếu sửa lỗi nóng cần phát hành gấp.

IV. Mô tả phần mềm cài đặt

a. Phạm vi trình bày

Báo cáo tập trung vào cách tổ chức mã nguồn, kiến trúc hệ thống, quy trình quản lý thay đổi cơ sở dữ liệu và kiểm thử tự động. Để đảm bảo an toàn thông tin, các dữ liệu và thông tin nhạy cảm của doanh nghiệp được lược bỏ hoặc thay thế bằng các ví dụ kỹ thuật phù hợp.

b. Môi trường và công cụ cần có

Tầng giao diện sử dụng nền tảng Angular với ngôn ngữ TypeScript, trong đó màn hình lễ tân vận hành trên Angular 9 kế thừa, còn màn hình quản lý mới được phát triển trên Angular 18. Phía máy chủ xây dựng trên nền tảng .NET và C#, truy xuất dữ liệu thông qua Entity Framework kết hợp các phương pháp tối ưu [6], lưu trữ chính trên hệ quản trị SQL Server có phân vùng dữ liệu theo khách hàng, sử dụng Redis làm bộ nhớ đệm và điều phối khóa phân tán [8], cùng Docker để đóng gói các dịch vụ độc lập. Quy trình biên dịch và kiểm thử được tự động hóa qua hệ thống tích hợp liên tục (Continuous Integration - CI). Về kiểm thử, phía máy chủ sử dụng thư viện xUnit, còn phía giao diện dùng Jasmine kết hợp Karma [10].

Sự song hành giữa hai phiên bản Angular phản ánh thực tế của một sản phẩm thương mại lớn: hệ thống hiện hữu cần được duy trì vận hành ổn định trong khi các phân hệ mới được tái thiết song song. Qua đó, em học được cách làm việc với cả mã nguồn kế thừa lẫn mã nguồn mới, đồng thời nắm bắt rõ hơn sự khác biệt giữa các thế hệ công cụ.

c. Cách tổ chức mã nguồn và cách dựng phần mềm

Hệ thống được tổ chức nhất quán theo kiến trúc phân tầng, với các lớp bên ngoài phụ thuộc vào lớp nghiệp vụ cốt lõi [1]. Tại mỗi dịch vụ phía máy chủ, mã nguồn chia thành các nhóm dự án độc lập: nhóm dùng chung chứa các kiểu dữ liệu và tiện ích nền tảng; nhóm trình bày chứa các API và đối tượng truyền dữ liệu (Data Transfer Object - DTO); nhóm ứng dụng chứa quy tắc nghiệp vụ, các lệnh thay đổi trạng thái và truy vấn; nhóm hạ tầng đảm nhiệm kết nối cơ sở dữ liệu và dịch vụ ngoài; cùng một dự án riêng

biệt cho kiểm thử tự động. Chiều phụ thuộc quy định lớp bên ngoài được biết về lớp bên trong, ngược lại thì không. Trong quá trình phát triển tính năng khóa phòng, em được trực tiếp tham gia hoàn thiện mã nguồn trên các tầng này, từ đó hiểu rõ hơn vai trò của việc phân tách trách nhiệm trong kiến trúc phần mềm.

Về quy trình dựng phần mềm, tầng giao diện tiến hành đồng bộ các gói phụ thuộc theo tệp khóa phiên bản nghiêm ngặt, nạp tệp cấu hình theo từng môi trường, sau đó biên dịch thành các tệp tĩnh triển khai lên máy chủ web. Riêng màn hình lễ tân còn có phiên bản đóng gói chạy trực tiếp trên máy tính tại quầy để kết nối máy in hóa đơn và thiết bị ngoại vi, đòi hỏi mọi thay đổi phải được kiểm thử trên cả hai môi trường. Phía máy chủ thực hiện khôi phục gói thư viện NuGet, biên dịch giải pháp, chạy toàn bộ kịch bản kiểm thử tự động và đóng gói thành phẩm triển khai.

d. Quản lý thay đổi cơ sở dữ liệu

Thay đổi cơ sở dữ liệu là khâu đòi hỏi tính cẩn trọng cao nhất, bởi sai sót mã nguồn có thể đưa về phiên bản trước dễ dàng, nhưng dữ liệu sai lệch trên môi trường sản xuất thì rất khó khắc phục nguyên trạng. Các thay đổi về cấu trúc như thêm bảng, bổ sung chỉ mục hoặc cập nhật thủ tục lưu trữ đều được quản lý tập trung trong một dự án riêng, giúp theo dõi lịch sử thay đổi và dễ dàng kiểm tra trước khi triển khai.

Đối với các thay đổi về dữ liệu, em xây dựng kịch bản dịch chuyển dữ liệu tự động và luôn chuẩn bị kèm kịch bản khôi phục (rollback) để có thể đưa dữ liệu về trạng thái trước đó khi xảy ra sự cố. Cách làm này được áp dụng trong cả đợt tách quyền hủy đơn và đợt cập nhật dữ liệu đa quốc gia. Việc chuẩn bị sẵn phương án khôi phục giúp giảm rủi ro khi triển khai và đảm bảo hệ thống có thể nhanh chóng khôi phục khi có vấn đề phát sinh.

e. Các môi trường và quy trình phát hành

Một thay đổi kỹ thuật thường trải qua quy trình phát hành gồm năm chặng. Lập trình viên hiện thực và tự kiểm tra trên nhánh cá nhân, sau đó tạo yêu cầu rà soát mã nguồn (code review) tới đồng nghiệp. Khi được phê duyệt, mã nguồn được hợp nhất vào nhánh phát triển chung nơi hệ thống CI tự động biên dịch và chạy kiểm thử hồi quy. Tiếp theo, sản phẩm được triển khai lên môi trường kiểm thử (staging) để bộ phận kiểm tra chất lượng (QA) xác nhận theo kịch bản người dùng thật, trước khi đưa vào nhánh chuẩn bị phát hành nhằm đóng băng phạm vi và kiểm tra lần cuối các tập lệnh dữ liệu. Chặng cuối cùng là phát hành chính thức lên hệ thống sản xuất theo lịch trình định sẵn, đi kèm cơ chế giám sát thời gian thực.

Đáng chú ý, giữa môi trường kiểm thử và nhánh phát triển luôn có chu trình phản hồi hai chiều: nếu phát hiện lỗi nghiệp vụ, phiếu công việc lập tức được chuyển lại kèm

thông tin chi tiết. Việc một lỗi được phát hiện và xử lý sớm tại môi trường kiểm thử giúp giảm chi phí xử lý sự cố và hạn chế ảnh hưởng đến người dùng cuối.

Ngoài ra, hai kinh nghiệm thực tế rất giá trị gồm: tính năng mới được mở dần cho từng nhóm khách hàng (phased rollout) thay vì phát hành đồng loạt cho toàn bộ người dùng; và trên môi trường web, việc gắn mã băm nội dung vào tên tệp tĩnh (cache busting) là cần thiết để ngăn trình duyệt người dùng giữ lại bộ đệm phiên bản cũ sau mỗi lần cập nhật.

f. Kiểm thử tự động trong quy trình

Trong thời gian thực tập, em đã đóng góp hơn 1.900 dòng mã kiểm thử tự động, phân bổ trên các tệp kiểm thử gồm các bài kiểm thử phía máy chủ và phía giao diện.

Tại phía máy chủ, các kịch bản kiểm thử tập trung vào logic nghiệp vụ cốt lõi: thuật toán sinh lệnh theo trạng thái thay đổi của tính năng khóa phòng, các quy tắc kiểm tra tính hợp lệ của dữ liệu, cơ chế thông báo theo ngôn ngữ và thuật toán tính toán thống kê lưu trú. Tên của từng trường hợp kiểm thử được cấu trúc theo hành vi nghiệp vụ rõ ràng (mô tả ngữ cảnh, hành động và kết quả kỳ vọng), giúp bộ kiểm thử trở thành tài liệu đặc tả hành vi trực quan của hệ thống.

Tại phía giao diện, em xây dựng các ca kiểm thử cho quy trình xác thực dữ liệu khi đặt phòng — nơi nhiều quy tắc nghiệp vụ cùng giao thoa: phòng đang khóa bảo trì, phòng có khách chưa thanh toán và khoảng đệm dọn buồng. Trường hợp kiểm thử em tâm đắc nhất là việc bảo toàn hành vi chặn đặt các phòng đang có khách chưa trả phòng mà không phụ thuộc vào khung thời gian yêu cầu. Mục đích chính của kiểm thử tự động không chỉ là chứng minh mã mới chạy đúng, mà là tạo cơ chế bảo vệ nhằm hạn chế các thay đổi trong tương lai gây lỗi hồi quy cho những nghiệp vụ hiện có.

V. Kết quả đạt được, hướng phát triển

a. Kỹ năng và kiến thức thu thập được

Những việc đã hoàn thành

Trong suốt thời gian thực tập, em đã tiếp nhận và hoàn thành hơn 170 phiếu công việc (bao gồm các yêu cầu tính năng nhỏ, tác vụ cải tiến và sửa lỗi) tương ứng với các đầu việc kỹ thuật trên các kho mã nguồn của sản phẩm. Các hạng mục đóng góp chính bao gồm:

- Tham gia phát triển tính năng khóa phòng bảo trì: từ mô hình dữ liệu, quy tắc nghiệp vụ, API phía máy chủ đến giao diện người dùng, bản dịch đa ngôn ngữ và viết kiểm thử tự động, đồng thời cập nhật công thức tính công suất phòng để loại

trừ phòng đang ngừng kinh doanh.

- Nghiên cứu và hiện thực hai chỉ tiêu thống kê mới về lượt khách nghỉ qua đêm và tổng số đêm nghỉ, đóng gói phần tính toán thành lớp xử lý thuần túy với các trường hợp kiểm thử tự động bao phủ các tình huống biên.
- Tái cấu trúc cơ chế phân quyền hủy đơn thành hai quyền độc lập, chuẩn bị tập lệnh chuyển đổi dữ liệu an toàn cho các cơ sở đang hoạt động.
- Các hạng mục tối ưu hiệu năng: giảm số lượt truy vấn và hạ độ phức tạp tính toán khi xuất tệp sổ quỹ, xử lý hiện tượng nghẽn tài nguyên cơ sở dữ liệu khi xuất hóa đơn chi tiết, bổ sung thời gian sống (TTL) cho khóa phân tán Redis và áp dụng quy ước tránh tính toán lặp trên giao diện.
- Xây dựng các phân hệ giao diện cho bản quản lý mới: trang tổng quan (dashboard) với các biểu đồ doanh thu, danh mục phòng, tài liệu hướng dẫn tích hợp kênh bán phòng trực tuyến và bảng tổng kết bàn giao ca.
- Địa phương hóa và đa ngôn ngữ: bổ sung tiếng Hàn, tiếng Trung, tiếng Lào cho các màn hình, xử lý đồng bộ định dạng ngày tháng, đơn vị tiền tệ và múi giờ, cùng các bài kiểm thử tự động trên cả hai tầng hệ thống.

Kiến thức kỹ thuật

Về cơ sở dữ liệu, em học được phương pháp phân tích câu truy vấn thực tế thay vì phụ thuộc vào mã sinh tự động từ ORM, nhận diện các điểm nghẽn phổ biến như phép nối bảng dư thừa, lọc trùng trên tập dữ liệu lớn hay nạp trước quan hệ không cần thiết. Em cũng tích lũy kinh nghiệm viết thủ tục lưu trữ tối ưu và xây dựng các kịch bản chuyển đổi dữ liệu an toàn cho môi trường sản xuất.

Về tối ưu hiệu năng, em hiểu rằng phần lớn giải pháp trong thực tế không đòi hỏi thuật toán phức tạp mà bắt nguồn từ những nguyên tắc rất cơ bản: hạn chế thực hiện lặp lại các phép tính tĩnh, không vận chuyển dữ liệu dư thừa qua mạng và thay thế quét tuần tự bằng tra cứu theo bảng băm.

Về kiến trúc phần mềm, em hiểu được giá trị của việc phân tầng và nguyên tắc phân định quyền sở hữu dữ liệu: trong hệ thống mà em tham gia, nhóm ưu tiên đặt logic tính toán nghiệp vụ tại dịch vụ sở hữu dữ liệu nguồn, tránh để cùng một chỉ tiêu bị tính toán độc lập ở nhiều nơi. Về kiểm thử tự động, nhận thức của em chuyển biến từ việc xem đây là thủ tục sang nhìn nhận kiểm thử như một bản đặc tả hành vi hệ thống và là cơ chế bảo vệ giúp hạn chế lỗi hồi quy khi mã nguồn thay đổi. Về đa ngôn ngữ, em nhận thức sâu sắc rằng địa phương hóa không chỉ là dịch chuỗi giao diện, mà bao gồm chuẩn hóa định dạng thời gian, số học, tiền tệ và đặc biệt là ranh giới múi giờ địa phương.

Kỹ năng làm việc

Bên cạnh kiến thức chuyên môn, kỳ thực tập đem lại cho em những kỹ năng làm việc quan trọng trong môi trường doanh nghiệp:

Kỹ năng đặt câu hỏi kỹ thuật: Thay vì mất nhiều thời gian tự tìm kiếm trong bế tắc, em hình thành phương pháp làm việc chủ động: sau một khoảng thời gian tự nghiên cứu và thử nghiệm hợp lý, nếu chưa tìm ra hướng đi thì chủ động trao đổi với người hướng dẫn kèm theo các phương án đã thử và giả thuyết cụ thể.

Kỹ năng viết nội dung commit: Từ thói quen viết nội dung commit chung chung, em dần hình thành cách mô tả rõ mã phiếu công việc và nội dung thay đổi, giúp tiết kiệm thời gian cho quá trình rà soát và truy vết mã nguồn sau này.

Phối hợp cùng bộ phận kiểm tra chất lượng (QA): Em hiểu rằng việc nhận phản hồi lỗi từ môi trường kiểm thử là một phần tất yếu của quy trình phát triển. Bộ phận QA tiếp cận hệ thống từ góc nhìn khách quan của người dùng thật, giúp phát hiện các tình huống biên mà lập trình viên dễ bỏ sót.

Sẵn sàng điều chỉnh giải pháp chưa tối ưu: Qua lần phải bỏ phần mã đã làm để chuyển logic sang đúng phân hệ, em hiểu rằng cần ưu tiên tính đúng đắn của kiến trúc hệ thống thay vì cố giữ lại giải pháp chỉ vì đã dành nhiều thời gian xây dựng.

Những bài học em cho là quan trọng nhất

1. Một tính năng chỉ thực sự hoàn thành khi nó hoạt động chuẩn xác và không gây ảnh hưởng tiêu cực đến các hành vi nghiệp vụ hiện có trên hệ thống.
2. Phạm vi của một thay đổi quan trọng tương đương nội dung của nó: với một lỗi cục bộ, cần ưu tiên giải pháp sửa đổi an toàn, có phạm vi hẹp thay vì vội vàng tái cấu trúc diện rộng khi chưa có kiểm thử bảo vệ.
3. Sử dụng ORM giúp tăng tốc độ phát triển nhưng không có nghĩa là có thể bỏ qua câu lệnh SQL thực tế được sinh ra; ở các tác vụ dữ liệu lớn, việc xem xét kế hoạch thực thi là cần thiết để phát hiện và xử lý các điểm nghẽn.
4. Quyền thao tác của người dùng phải được kiểm tra ở phía máy chủ, thay vì chỉ ẩn hoặc vô hiệu hóa nút trên giao diện; các ràng buộc trên giao diện chỉ hỗ trợ trải nghiệm người dùng chứ không thể thay thế cho việc kiểm soát quyền thực tế.
5. Kiểm thử tự động có giá trị lớn nhất trong việc bảo vệ hành vi hiện hữu của hệ thống; các ca kiểm thử giúp em tự tin hơn khi sửa đổi mã nguồn và hạn chế tối đa lỗi hồi quy phát sinh.

Những điều em chưa làm được

Nhìn nhận một cách khách quan, em nhận thấy bản thân còn một số hạn chế cần tiếp tục hoàn thiện. Năng lực ước lượng thời gian thực hiện công việc đôi khi chưa sát với thực tế, chủ yếu do chưa lường hết thời gian cần thiết để đọc hiểu mã nguồn kế thừa. Việc viết kiểm thử tự động chưa thực sự đồng đều giữa các tầng: em tập trung nhiều vào logic nghiệp vụ phía máy chủ nhưng còn hạn chế ở tầng hiển thị giao diện. Em vẫn cần rèn luyện thêm khả năng tự chất vấn đa chiều các trường hợp ngoại lệ thay vì trông đợi vào sự định hướng của người hướng dẫn. Cuối cùng, mức độ hiểu biết của em về hệ thống mới tập trung sâu vào các phân hệ trực tiếp đảm nhiệm chứ chưa bao quát toàn diện mọi nhánh dịch vụ của sản phẩm.

b. Hướng phát triển tiếp theo

Đối với tính năng khóa phòng bảo trì, hướng phát triển tiếp theo là mở rộng cơ chế khóa phòng lặp lại theo chu kỳ định kỳ tự động và liên kết với kế hoạch quản lý tài sản, giúp chủ cơ sở theo dõi toàn diện lịch sử sửa chữa của từng phòng; mô hình sinh lệnh theo trạng thái thay đổi đã xây dựng có thể tái sử dụng trực tiếp cho yêu cầu này.

Đối với phân hệ thống kê lưu trú, giải pháp nâng cao hiệu năng trong dài hạn là áp dụng cơ chế tổng hợp trước (pre-aggregation) theo từng ngày và lưu vào bảng tổng hợp đệm; khi người dùng tra cứu khoảng thời gian tùy chọn, hệ thống chỉ cần cộng dồn các bản ghi ngày đã xử lý mà không phải quét lại toàn bộ dữ liệu gốc. Với công tác tối ưu hiệu năng, hướng đi tiếp theo là thiết lập hệ thống giám sát hiệu năng ứng dụng (Application Performance Monitoring - APM) để phát hiện sớm các truy vấn chậm trước khi phát sinh phản ánh từ khách hàng.

Về định hướng cá nhân, em đặt mục tiêu tiếp tục phát triển theo hướng kỹ sư toàn diện có khả năng bao quát cả tầng giao diện lẫn tầng máy chủ. Ba mục tiêu cụ thể em tập trung theo đuổi gồm: nghiên cứu chuyên sâu về tối ưu chỉ mục và kế hoạch thực thi câu truy vấn cơ sở dữ liệu; rèn luyện phương pháp phát triển hướng kiểm thử (Test-Driven Development - TDD) cho các phân hệ nghiệp vụ phức tạp; và nâng cao năng lực phân tích tác động toàn diện trước khi bắt tay vào hiện thực mã nguồn.

Kỳ thực tập tại doanh nghiệp không chỉ củng cố kiến thức học đường, mà quan trọng hơn đã trang bị cho em phương pháp làm việc chuẩn mực trên một hệ thống thương mại thực tế đang vận hành và phục vụ hàng nghìn người dùng mỗi ngày.

Tài liệu tham khảo

[1] Robert C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017.

[2] Michael Feathers, *Working Effectively with Legacy Code*, Prentice Hall, 2004.

[3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, 4th Edition, The MIT Press, 2022.

[4] Martin Fowler, *CQRS (Command Query Responsibility Segregation)*, martinfowler.com/bliki/CQRS.html, 2011.

[5] Google, *Angular Documentation: Change Detection and Custom Pipes*, angular.dev.

[6] Microsoft, *Entity Framework Documentation and Performance Practices*, learn.microsoft.com.

[7] Microsoft, *SQL Server Transaction Locking and Execution Plans Guide*, learn.microsoft.com.

[8] Redis Ltd., *Distributed Locks with Redis and Key Expiration Pattern*, redis.io.

[9] Mozilla Developer Network (MDN), *Unicode Character Class Escape in Regular Expressions*, developer.mozilla.org.

[10] xUnit.net and Jasmine Contributors, *Unit Testing Guidelines and Test Case Organization*, xunit.net, jasmine.github.io.

KiotViet

Ý kiến đánh giá:

Tinh thần: Nghiêm túc, trách nhiệm, chủ động trong công việc và có tinh thần hỗ trợ đồng đội.....

Kiến thức: Có nền tảng chuyên môn tốt, tiếp thu nhanh và biết vận dụng kiến thức vào thực tế.....

Khả năng: Tư duy xử lý vấn đề tốt, chủ động tìm giải pháp và có khả năng đảm nhận công việc có độ phức tạp.....

Đánh giá: Hoàn thành tốt công việc, có tinh thần cầu tiến và đóng góp.

Mức đánh giá tổng thể: 10/10.....

Hà Nội, ngày 23 tháng 9 năm 2026.

Xác nhận của Công ty
(Ký tên, đóng dấu)

M.S.D.N: 0109... CÔNG TY CỔ PHẦN CÔNG NGHỆ KIOTVIET THÀNH PHỐ

GIÁM ĐỐC NHÂN SỰ
Nguyễn Thị Kim Oanh

Người hướng dẫn
(Ký, ghi rõ họ tên)

Nguyễn Mạnh Cường.

Ý kiến đánh giá:

.....

Sinh viên có ý thức tốt.....

.....

.....

.....

.....

.....

.....

Điểm số: 9 Điểm chữ: Chín

Hà Nội, ngày 24 tháng 9 năm 2026

Giảng viên đánh giá

(Ký, ghi rõ họ tên)

Trần Mạnh Cường